

KV layout and DRAM row locality for LLM decode on an NPU

ONNXim + Ramulator2 • measured results, 2026-09-13

Scope: LLaMA-2 7B one transformer block, ONNXim + Ramulator2, HBM3 16 ch / 2 pch / 4 bg / 4 bank, 1 KB rows, 32 B requests, 819.2 GB/s. Default config `_c128.json` (4 x 128x128 cores) unless noted. Workload `az128` = 128 real requests from the Azure LLM Inference Dataset, 200,064 cached tokens, one decode step (KV = 90% of DRAM traffic). All numbers below are post-2026-09-09, with the operand-tagged stream split and `ONNXIM_ICNT_PORT_BY_CHANNEL=1`.

Current state in four lines. KV layouts form one axis: bytes of a single head kept contiguous. Row-buffer locality peaks when that unit is exactly one DRAM row-stripe; a bank-isolated layout (**headbank**) reaches 92% KV row hit with 61% fewer activations than vLLM's default. The runtime benefit is mapper-dependent (1.36x under RoBaRaCoCh, ~1.01x under XOR hashing); the locality and activation-energy benefit is not. Head-major is the **bandwidth** end of the axis, not the locality end.

`FINDINGS.md` is the long-form history including pre-fix measurements; §14 there lists what is superseded. This file is the current state.

1. One axis: bytes of a head kept contiguous

Every KV layout is the same flattening with one parameter U = the per-head contiguous unit:

$$\text{addr}(\text{head}, p) = (p / U) \times (\text{NH} \times U) + \text{head} \times U + (p \bmod U)$$

where p = the head's own stream position in bytes and $\text{NH} = 32$ KV heads. One token of one head is `head_dim` x `precision` = 256 B, so $U = \text{block_size} \times 256 \text{ B}$:

U	block size	layout	per-head unit vs a 16 KB row-stripe
4 KB	16 (vLLM default)	block-major	1/4 row – four heads share every row
16 KB	64	headbank K=1	exactly one row
32 KB	128	headbank K=2	two rows
whole context	–	head-major	head index sits above the row field

Head-major is the $U \rightarrow \infty$ limit ($p < U$, so the outer term vanishes and $\text{addr} = h \times \text{ctx} + p$). Block-64 and headbank K=1 are the same addresses reached two ways: measured 23,223,415 vs 23,214,626 cycles, 0.04% apart on a deterministic simulator.

****Practical form: this is vLLM's `-block-size` flag.**** Going from 16 to 64 is most of the benefit, with no new layout. The equivalence is model- and precision-dependent (' $U = B \times \text{head_dim} \times \text{precision}$ '); "one row-stripe per head" is the portable statement.

2. Headbank: how the mapping works

Under RoBaRaCoCh, ONNXim hashes the channel out of the low bits and Ramulator slices the rest. In global address bits for this config:

```
bits [0:5)   byte within the 32 B request
bits [5:9)   channel (hashed; 16 channels)
bits [9:14)  column    -> 32 requests x 32 B = one 1 KB row per channel
bits [14:19) bank slot -> pch(1) + bankgroup(2) + bank(2) = 32 banks per channel
bits [19:)   row
```

Two constants follow: **16 KB** = 2^{14} is one row in **every** channel (a 16 KB run scatters 512 requests, 32 per channel, differing only in column), and **512 KB** = 2^{19} = 32 such stripes, one per bank slot. So with $U = 16 \text{ KB}$ the layout formula **is** a bit-packing:

$$\text{addr} = c \times 2^{19} + h \times 2^{14} + \text{within} \quad (\text{within} < 2^{14}, h < 32)$$

|- row -| |- bank -| |- column + channel -|

The head index is placed directly on the bank field. Nothing in Ramulator changes.

Worked example – head 5, token 100, dim 0, block 16 (logical block 6, offset 4):

```
p      = (6 x 16 + 4) x 256 B = 25,600
c      = 25,600 / 16,384 = 1          within = 9,216
addr   = 1 x 524,288 + 5 x 16,384 + 9,216 = 615,424 = 0x96400
        bits [14:19] = 5 -> bank slot 5 = the head index
        bits [19:]   = 1 -> row 1      = the chunk index
```

Continuing: token 164 -> bank 5 row 2; token 1,562 -> bank 5 row 24. Head 6 -> bank 6, same rows.

Two distinct benefits, which fail independently:

- *The row is full of one head.* All 32 request slots of every row visit serve the head that opened it. vLLM's 4 KB unit fills 8 of 32, so three quarters of each activation is data for heads you are not reading. This needs only $U \geq 16$ KB.
- *The head owns the bank.* Head 5's rows are all in bank 5 and no other stream can close them. This needs the *alignment* – U exactly 2^{14} and the tensor base padded to 512 KB. Get U right and the base wrong and you keep (1), lose (2).

Generalisation. $U = K \times 16$ KB gives each head K banks: $\text{bank} = (\text{h} \times K + \text{c} \bmod K + \text{tile} \times \text{stride}) \bmod 32$, with the row field carrying $\text{h} / (32/K)$ to separate heads that now share a slot. K is the isolation/streaming dial (§5).

Geometry constants are parameters, not literals (ONNXIM_KV_BANKCHUNK, _NBANKS): the 48-sub-channel LPDDR6 config needs 128 KB and 16 banks.

3. Measured: az128, 4 x 128x128

layout	weights hit	KV hit	KV miss	KV confl	ACT/ch	dram cycles	avg BW	while-busy
head-major	74.1%	70.0%	22.1%	7.9%	2.31M	17.68M	86.3%	98.5%
block-major b16 (vLLM)	74.0%	79.4%	4.6%	12.0%	1.70M	24.13M	63.3%	68.7%
headbank K=1	74.0%	91.9%	3.2%	4.9%	0.90M	23.21M	65.8%	71.5%
headbank K=2	73.9%	83.2%	11.9%	4.9%	1.46M	18.88M	80.9%	90.2%
K=2 + V-shift + block 64	73.9%	82.2%	14.6%	3.2%	1.53M	18.12M	84.2%	94.4%

Head-major is 1.36x faster than vLLM's layout. **It is not because of row locality** – it has the lowest KV hit in the table and does the *most* activations. It wins because (a) its non-hits are cheap misses rather than conflicts (7.9% vs 12.0%: a conflict is PRE+ACT on the critical path with the bank's queue stalled, a miss is ACT on an idle bank), and (b) its stream hops banks every row, so the next row's activate overlaps the current row's data – 98.5% while-busy against block-major's 68.7%. The original §1 claim that head-major wins on locality was an artifact of the head-0 bug.

Block-major's 79.4% comes from *sharing*: a row holds four heads' 4 KB slices, and the four cores process heads $h..h+3$ concurrently, so one activation can serve four cores. It pays for that with conflicts whenever they drift apart, and with 8-requests-per-activation streams that are tFAW-bound ($4 \text{ ACT} / 49 \text{ cycles} \times 8 \times 32 \text{ B} \sim 63\%$ of the bus; measured 68.7%).

Headbank needs no sharing: 92% hit, conflicts down 40%, activations halved. Its cost is intra-stream overlap (§5). **K=2 + V-shift + block 64 is the best joint point:** +12 points of KV hit and 34% fewer activations than head-major, at 2.5% runtime.

4. What the miss/conflict split means

A bank has one row buffer. The cost model that explains every row of every table:

```
hit      CAS only
miss     bank closed -> ACT (~23 cyc), the PRE was already paid off the critical path
conflict wrong row open -> PRE + ACT (~46 cyc), queue behind it stalls
```

So row-hit rate alone does not order the layouts by speed; hit + conflict-vs-miss + stream overlap does. Reporting KV hit without the conflict column is how the pre-fix conclusions went wrong.

5. The K dial, and when isolation stops working

Isolation requires **concurrent streams** $\leq$ **banks**. Streams = cores x tile_depth (12 at 4 cores, 48 at 16 cores) plus whatever the scheduler pulls from the next request.

config (streams/banks)	head-major	block-major	K=1	K=2
1 core, 8 req (3/32)	78.4%	76.7%	92.0%	93.8%
4 cores, 8 req (12/32)	70.4%	79.4%	91.7%	82.6%
4 cores, az128 (12/32)	70.0%	79.4%	91.9%	83.2%
16 cores, az128 (48/32)	74.1%	68.8%	66.3% (broken)	73.3%

Why K=1 costs bandwidth: a bank-confined stream pays $t_{RCD} + 64 \text{ data} + t_{RTP} + t_{RP} \sim 117$ cycles per 64 cycles of data, $\sim 55\%$ single-bank duty, and only *another bank* can fill the gap – either its own ($K \geq 2$) or another head's. At 16 cores heads must share banks, isolation is impossible, and K=1 falls below head-major.

The controller cannot fix this. A scheduler tier issuing PRE/ACT as early as timing allows cut the measured PRE->ACT gap from 62 to 28.5 cycles and changed runtime by 127 cycles out of 7.4M at 1 core, and was 3% *worse* at 4 cores. When every queued request targets a bank mid-row-switch there is nothing to reorder; the remaining terms are DRAM timings.

6. Mapper dependence: XOR hashing equalises the layouts

RoBaRaCoChXOR folds row bits into the bank field. On az128:

layout	RoBaRaCoCh	XOR
head-major	70.0% hit, 17.68M	70.1%, 17.66M
block-major b16	79.4%, 24.13M	67.8%, 17.93M
headbank K=2	83.2%, 18.88M	69.9%, 17.67M

All three converge to ~ 17.7 M cycles and $\sim 70\%$ hit. Hashing hands block-major the bank parallelism it lacked (68.7% -> 95.6% while-busy) at the cost of its locality, and it *destroys* headbank by scrambling the head->bank assignment. ****The runtime claim is therefore mapper-dependent; the locality/activation claim is not**** – no hashed mapper exceeds $\sim 70\%$ KV hit, and headbank reaches 92%.

A headbank-aware controller must leave bits [14:19] alone (hash only the row bits above them).

7. The weight stream

Unchanged by any KV layout (74% hit in every row of §3) and re-measured as correct:

- Tile-major swizzle (ONNXIM_WEIGHT_SWIZZLE=1) is the large lever: row hit 73.6% -> 95.7%, while-busy BW 90.6% -> 98.1%. Validated on real DDR5 silicon, same bytes and access count, order only: 15.8 -> 46.2 GB/s (2.92x).
- Concurrency, not layout, costs the rest: **96.3% at 1 core, 74% at 4** – four cores' tiles colliding in banks chosen by address position.
- Bank-isolating weights the same way was **measured and is inert**: 73.9% -> 73.1%, conflicts 4.6% -> 3.5%, runtime -0.2%. The 4-core loss is 21.5% *misses* with only 4.6% conflicts (a stream's own row switches), and head-major already serves a full queue at 99.4%. No headroom.
- Weights are read **once per decode step for the whole batch** (405 MB, identical at batch 1, 8 and 128). Only KV scales with the batch. That asymmetry is why layout matters for KV and not weights.

8. Dual-roofline model (unchanged, and it transfers across DRAM technology)

```
memory floor = weight_bytes / BW (405 MB; scales with precision)
array floor  = weight_bytes / (core_width x precision x num_cores) (no DRAM term)
```

Predicted every batch-1 result to 4-5%. New evidence: on llama_dec1 at 1 core the runtime is identical on HBM3, LPDDR6-10667 and LPDDR6-14400 (1,655,575 / 1,657,147 / 1,656,580 cycles) – the array floor binds, and neither a 1.3x bandwidth difference nor a 2.9x latency difference reaches it.

9. LPDDR6 vs HBM3

LPDDR6.cpp = one JESD209-6 sub-channel (12 DQ, 4 bg x 4 bank, BL24 = 32 B data + 4 B meta). _lp6_48ch_{10667,14400}.j = 48 sub-channels (576-bit, Apple M4-Max class), payload peak 683 / 922 GB/s, bracketing one HBM3 stack.

Core timings are not public: LPDDR5X-class values in ns, frequency-scaled (the HBM3_6.4Gbps.yaml method). Relative comparison only.

Gates (standalone bin/ramulator2): unloaded read latency HBM3 15.9 ns (= tRCD+tCL+nBL+1 exactly) vs LPDDR6 45.9 / 45.0 ns; sequential streaming HBM3 2.00 cycles/RD (nominal), LPDDR6 13.1 (92% of nominal – the WCK CAS-sync).

llama_dec1, 4 cores	HBM3	LPDDR6-10667	LPDDR6-14400
core cycles	532,739	757,698 (+42%)	565,583 (+6%)
delivered BW	96.9% of nominal	77.4%	76.4%
while-busy	99.5%	85.8%	85.4%
weights hit / ACT per ch	74.9% / 198k	92.3% / 20k	94.2% / 15k

The floor model predicted +20% / -11% from bandwidth alone; the extra is the LPDDR5X-derived model serving a full queue at only ~86%. Report against effective peak and revisit with vendor timings. LPDDR6's larger sub-channel rows give far better locality (92-94% weights hit, 10x fewer ACTs/ch).

Energy (scripts/dram_energy.py, command counters x literature IDD-class constants): HBM3 16.9 mJ = 5.00 pJ/bit; LPDDR6-10667 19.8 mJ = 5.84 pJ/bit. LPDDR6 does 2-3x fewer activations, so the comparison rides entirely on the interface term: ****LPDDR6 wins iff its IO energy is below 3.2 pJ/bit**** (HBM3 assumed 2.5). That inequality is the citable result, not the 1.17x.

10. Upstream ONNXim/Ramulator bugs found (8)

bug	effect	file
kv_head_idx = head_idx / _nkvh	every attention head read KV head 0	Attention.cc
tile-local seq_idx	contexts > 2730 re-read chunk 0	Attention.cc
KV-concat tiles skip	KV cache never written	KVCacheConcat.cc
value_out_address from key_cache_tensor	V writes landed on K	KVCacheConcat.cc
acc-spada rotation vs last-issued tile	flushed a bank a live tile still used -> silent throw	Core.cc
ipoly_hash_function exit(1) for channels != 16/32/64	every 48-channel run died with a clean log	Hashing.cc / Dram.cc
log2(cols) - log2(prefetch) as one float	truncated for non-power-of-two prefetch	Dram.cc
dram_nbl hard-coded 1; exact-match command names	BW print 2x low; LPDDR RD32/ACT-2 counted as 0	Dram.cc, controller

Plus ReadWriteTrace inverting R/W and looping forever (use LoadStoreTrace).

All are silent, timing-dependent, and invisible to count-based validation – the same class as the first three. Counting addresses cannot catch a wrong index.

11. Measurement pitfalls (both cost real time)

- The controller's "kv+act" class was an address threshold, so activation traffic (37% row hit) was averaged into KV. Requests now carry `Instruction::operand_id` through to `Request::source_id` and the controller prints three ROWSPLIT lines. K=1's true KV hit is 91.9%, not the 87.5% the mixed class reported.
- **Trace scripts must use the full `RAMULATOR_WEIGHT_LIMIT`** (500,000,000). The CSV `address` column is global; only the controller sees the /16 compacted form. Applying the compacted threshold to the CSV classes 97% of the weight stream as KV – hours of "KV stream" analysis were the weight stream, and three layout knobs were built against a phantom. Use the `operand` column.

12. Negative results (measured; do not re-run)

RoCoBaCh mapper (bank bits below column -> tFAW-bound, 2x slower); `tile_depth` 6 (+1%); request-parallel (1.6x slower); V-bank shift alone, activation lane, request rotation, M-tile stride (neutral once request ordering was fixed); bank-prep scheduling (§5); weight tile-bank isolation (§7); KV block size for *head-major* (inert, +2.5 points over 16x); speculative decoding is layout-insensitive (head vs K=2 within 0.6%; KV is 7% of that workload).

13. Tools and knobs

scripts/: `tops_visible.py` (vLLM sim -> trace), `vllm_trace.py`, `gen_trace.py`, `specdec_trace.py` / `specdec_analyze.py`, `analyze_interleave.py`, `dram_energy.py`, `replay_trace.py`.

Traces: `out/traces_layout/az128_{head,headbank_k1,block}.csv.gz` – 122,117,889 rows each, same bytes, arrival-order, with `cycle` (DRAM cycles @3.2 GHz) and `operand` (100 Q, 101 K/weight, 102 V, >=200 output, 0 KV write).

Env vars, all default to stock: `ONNXIM_WEIGHT_SWIZZLE`, `ONNXIM_INTRA_ORDER`, `ONNXIM_KV_BLOCK/_LAYOUT/_POOL/_ALLOC/_UN`, `ONNXIM_KV_LAYOUT=headbank + _BANKS_PER_HEAD / _V_BANK_OFFSET / _TILE_BANK_STRIDE / _REQ_ROTATE / _BANKCHUNK / _NBANKS`, `ONNXIM_ACT_LANE`, `ONNXIM_WEIGHT_TILEBANK`, `ONNXIM_PAR_STRATEGY`, `ONNXIM_ICNT_PORT_BY_CHANNEL`, `ONNXIM_TILE_LOG`, `ONNXIM_DRAM_TRACE/_LIMIT/_OCC`, `RAMULATOR_REQBUF/ACTBUF/RETRY`, `RAMULATOR_SCHED_ROWBIT/_STARVE`, `RAMULATOR_WEIGHT_LIMIT`.

`ONNXIM_ICNT_PORT_BY_CHANNEL=1` is worth defaulting on: stock fans one core FIFO across 16 ports and the interconnect drains a channel round-robin over them, locally reordering a channel's arrivals. Conflicts fell 35-45% on every layout.

14. Still resting on pre-fix numbers (re-run before citing)

The six-workload layout table, the GQA/MQA arms of the mapper ablation, the vLLM-LIFO allocation study, and the block-size sweep's absolute values (shape holds; it predates the operand split).

15. The claim

*KV layouts for paged attention form a single axis – bytes of one head kept contiguous. Row-buffer locality peaks when that unit equals one DRAM row-stripe, where a bank-isolated layout reaches 92% KV row hit and 61% fewer activations than vLLM's default block size. The runtime benefit is mapper-dependent (1.36x under RoBaRaCoCh, ~1.01x under XOR bank hashing); the locality and activation-energy benefit is not. Head-major – the opposite end of the same axis – is fastest but has the *worst* row locality, winning on bank-level parallelism instead.*

Prior art: head-grouped KV is known for NAND flash (KVNAND) and as a compression theme (HeadKV, AdaKV); no prior work does the DRAM row-buffer version, the block-size equivalence, or the mapper ablation. CAUTION: the related-work PDFs were assessed through summarisers, never read.